

```
!apt-get update -qq
!pip install -q scikit-image opencv-python matplotlib numpy scipy pandas
```

W: Skipping acquire of configured file 'main/source/Sources' as repository '<https://r2u.stat.illinois.edu/ubuntu>

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage import util, filters, morphology, exposure
from skimage.metrics import peak_signal_noise_ratio as PSNR, structural_similarity as SSIM
from scipy.ndimage import median_filter
from scipy.signal import convolve2d
import pandas as pd
from skimage.feature import canny

# small display helper
def show_pair(title, imgA, imgB, labelA="A", labelB="B", cmap='gray'):
    plt.figure(figsize=(10,4))
    plt.suptitle(title)
    plt.subplot(1,2,1); plt.imshow(imgA, cmap=cmap); plt.title(labelA); plt.axis('off')
    plt.subplot(1,2,2); plt.imshow(imgB, cmap=cmap); plt.title(labelB); plt.axis('off')
    plt.show()

# Add noise functions
def add_salt_pepper(img, amount=0.05):
    return util.random_noise(img, mode='s&p', amount=amount)

def add_speckle(img, var=0.04):
    return util.random_noise(img, mode='speckle', var=var)

def add_gaussian(img, var=0.01):
    return util.random_noise(img, mode='gaussian', var=var)

# alpha-trimmed mean filter (2D)
def alpha_trimmed_mean(img, size=3, d=2):
    # d = number of extreme pixels to trim (total); must be even and < size*size
    pad = size//2
    padded = np.pad(img, pad, mode='reflect')
    out = np.zeros_like(img)
    m = size*size
    trim = int(d//2)
    for i in range(out.shape[0]):
        for j in range(out.shape[1]):
            block = padded[i:i+size, j:j+size].flatten()
            block_sorted = np.sort(block)
            trimmed = block_sorted[trim: m-trim]
            out[i,j] = np.mean(trimmed)
    return out

# compute edge preservation score: overlap ratio of Canny edges
def edge_overlap(ref, proc, sigma=1.0):
    ref_edges = canny(ref, sigma=sigma)
    proc_edges = canny(proc, sigma=sigma)
    # overlap = intersection / max(ref_edges.sum(),1)
    inter = np.logical_and(ref_edges, proc_edges).sum()
    denom = max(ref_edges.sum(), 1)
    return inter/denom

# convert images to float [0,1] and back helpers
def to_float(img):
    if img.dtype == np.uint8:
        return img.astype(np.float32)/255.0
    else:
        return img.astype(np.float32)

def to_uint8(img):
    i = np.clip(img*255.0, 0, 255).astype(np.uint8)
    return i
```

```
# paths you provided
image_paths = [
    "/content/Picture1.png",
    "/content/Picture2.png",
    "/content/Picture3.png",
    "/content/Picture4.png",
    "/content/Picture5.png"
]
```

```

images = {}
for p in image_paths:
    img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
    if img is None:
        print("Warning: could not load", p)
    else:
        images[p] = to_float(img) # store float [0,1]
        print("Loaded:", p, "shape:", img.shape)

# If none loaded, raise
if not images:
    raise RuntimeError("No images loaded – check paths.")

```

```

Loaded: /content/Picture1.png shape: (385, 345)
Loaded: /content/Picture2.png shape: (385, 262)
Loaded: /content/Picture3.png shape: (308, 618)
Loaded: /content/Picture4.png shape: (300, 625)
Loaded: /content/Picture5.png shape: (383, 329)

```

```

import numpy.fft as fft

def noise_heuristics(img):
    # crude heuristics: detect salt&pepper by many near-0/1 pixels,
    # speckle by variance proportional pattern, impulsive by large outliers.
    flat = img.flatten()
    # salt-pepper heuristic: fraction of pixels very close to 0 or 1
    sp_frac = np.mean((flat < 0.01) | (flat > 0.99))
    # speckle heuristic: high variance relative to local mean
    # Corrected 'selem' to 'footprint' for windowed_histogram
    local_std = filters.rank.windowed_histogram((img*255).astype(np.uint8), footprint=np.ones((7,7)))
    var = np.var(img)
    # spectral check for tonal impulses: look for sharp FFT peaks
    F = np.abs(fft.rfft(flat - np.mean(flat)))
    F = F / (np.max(F)+1e-12)
    peakiness = np.max(F) # if extremely close to 1 -> strong tone
    return {"salt_pepper_frac": sp_frac, "var": var, "fft_peak": peakiness}

print("Noise heuristics (per image):")
for p,img in images.items():
    h = noise_heuristics(img)
    print(p, "| sp_frac={:.3f} var={:.4f} fft_peak={:.3f}".format(h["salt_pepper_frac"], h["var"], h["fft_peak"]))

```

```

Noise heuristics (per image):
/content/Picture1.png | sp_frac=0.013 var=0.0820 fft_peak=1.000
/content/Picture2.png | sp_frac=0.035 var=0.0726 fft_peak=1.000
/content/Picture3.png | sp_frac=0.001 var=0.0487 fft_peak=1.000
/content/Picture4.png | sp_frac=0.000 var=0.0168 fft_peak=1.000
/content/Picture5.png | sp_frac=0.000 var=0.0409 fft_peak=1.000

```

```

# parameters: adjust as needed
snp_amount = 0.06 # salt & pepper fraction
speckle_var = 0.03
gauss_var = 0.005

noisy_versions = {} # structure: noisy_versions[image_path] = { 's&p':..., 'speckle':..., 'gauss':... }

for p,img in images.items():
    s_p = add_salt_pepper(img, amount=snp_amount)
    sp = add_speckle(img, var=speckle_var)
    g = add_gaussian(img, var=gauss_var)
    noisy_versions[p] = {'s&p': s_p, 'speckle': sp, 'gaussian': g}
    # show small preview
    print("Created noisy versions for:", p)

```

```

Created noisy versions for: /content/Picture1.png
Created noisy versions for: /content/Picture2.png
Created noisy versions for: /content/Picture3.png
Created noisy versions for: /content/Picture4.png
Created noisy versions for: /content/Picture5.png

```

```

# window sizes to test for median and alpha-trim
median_sizes = [3,5,7,9]
alpha_sizes = [3,5,7] # alpha-trim window sizes
alpha_d = 2 # number of trimmed pixels (d). For size 5, d=2 removes 1 from each end.

# morphological structuring elements sizes
morph_sizes = [3,5,7]

# results table rows
rows = []

for p, variants in noisy_versions.items():
    orid = images[p]

```

```

for noise_name, noisy in variants.items():
    print("\n--- Image:", p, "| Noise:", noise_name)
    # evaluate baseline
    base_psnr = PSNR(orig, noisy, data_range=1.0)
    base_ssim = SSIM(orig, noisy, data_range=1.0)
    base_eov = edge_overlap(orig, noisy)
    print("Baseline PSNR: {:.2f}, SSIM: {:.4f}, EdgeOverlap: {:.4f}".format(base_psnr, base_ssim, base_eov))

    # MEDIAN filters
    for k in median_sizes:
        med = median_filter(noisy, size=k)
        ps = PSNR(orig, med, data_range=1.0)
        ss = SSIM(orig, med, data_range=1.0)
        eo = edge_overlap(orig, med)
        rows.append({
            "image": p, "noise": noise_name, "method": f"median_{k}", "psnr": ps, "ssim": ss, "edge_overlap": eo
        })
    # print a few values
    print(f" median {k}: PSNR={ps:.2f}, SSIM={ss:.4f}, EdgeOv={eo:.4f}")

    # ALPHA-TRIMMED mean
    for k in alpha_sizes:
        at = alpha_trimmed_mean((noisy*255).astype(np.uint8), size=k, d=alpha_d)
        at_f = at/255.0
        ps = PSNR(orig, at_f, data_range=1.0)
        ss = SSIM(orig, at_f, data_range=1.0)
        eo = edge_overlap(orig, at_f)
        rows.append({
            "image": p, "noise": noise_name, "method": f"alpha_trim_{k}_d{alpha_d}", "psnr": ps, "ssim": ss, "edge_overlap": eo
        })
    print(f" alpha-trim {k}: PSNR={ps:.2f}, SSIM={ss:.4f}, EdgeOv={eo:.4f}")

    # MORPHOLOGICAL (opening then closing and vice-versa) - good for impulse/salt-pepper
    for s in morph_sizes:
        selem = morphology.square(s)
        # opening then closing
        opened = morphology.opening((noisy*255).astype(np.uint8), selem)
        closed = morphology.closing(opened, selem)
        mc = closed.astype(np.float32)/255.0
        ps = PSNR(orig, mc, data_range=1.0)
        ss = SSIM(orig, mc, data_range=1.0)
        eo = edge_overlap(orig, mc)
        rows.append({
            "image": p, "noise": noise_name, "method": f"morph_openclose_{s}", "psnr": ps, "ssim": ss, "edge_overlap": eo
        })
    print(f" morph {s}: PSNR={ps:.2f}, SSIM={ss:.4f}, EdgeOv={eo:.4f}")

    # show example visual for the best median size (highest SSIM among medians)
    med_candidates = []
    for k in median_sizes:
        med = median_filter(noisy, size=k)
        ss = SSIM(orig, med, data_range=1.0)
        med_candidates.append((ss, k, med))
    med_candidates.sort(reverse=True, key=lambda x: x[0])
    best_med = med_candidates[0]
    print(f" Best median size (by SSIM) = {best_med[1]} (SSIM={best_med[0]:.4f})")
    show_pair(f"{p} | {noise_name} : Original vs BestMedian{best_med[1]}", orig, best_med[2], "Original", f"BestMedian{best_med[1]}")

# Create dataframe
df = pd.DataFrame(rows)
df_sorted = df.sort_values(['image', 'noise', 'psnr'], ascending=[True, True, False])
display(df_sorted.head(40))

```



```

--- Image: /content/Picture1.png | Noise: s&p
Baseline PSNR: 16.83, SSIM: 0.4157, EdgeOverlap: 0.6161
median 3: PSNR=27.32, SSIM=0.8325, EdgeOv=0.6396
median 5: PSNR=23.80, SSIM=0.6127, EdgeOv=0.2810
median 7: PSNR=22.63, SSIM=0.4818, EdgeOv=0.1347
median 9: PSNR=22.17, SSIM=0.4240, EdgeOv=0.0839
alpha-trim 3: PSNR=25.94, SSIM=0.7781, EdgeOv=0.6355
alpha-trim 5: PSNR=23.65, SSIM=0.5775, EdgeOv=0.2676
alpha-trim 7: PSNR=22.62, SSIM=0.4494, EdgeOv=0.0963
morph 3: PSNR=22.92, SSIM=0.6404, EdgeOv=0.3098
morph 5: PSNR=19.31, SSIM=0.3946, EdgeOv=0.0948
morph 7: PSNR=14.95, SSIM=0.2768, EdgeOv=0.0753
/tmp/ipython-input-4145038815.py:48: FutureWarning: `square` is deprecated since version 0.25 and will be removed in a future version
selem = morphology.square(s)
Best median size (by SSIM) = 3 (SSIM=0.8325)

```

/content/Picture1.png | s&p : Original vs BestMedian3



```

--- Image: /content/Picture1.png | Noise: speckle
Baseline PSNR: 21.54, SSIM: 0.6297, EdgeOverlap: 0.7454
median 3: PSNR=25.08, SSIM=0.6613, EdgeOv=0.5833
median 5: PSNR=23.33, SSIM=0.5339, EdgeOv=0.2604
median 7: PSNR=22.43, SSIM=0.4421, EdgeOv=0.1088
median 9: PSNR=22.04, SSIM=0.3997, EdgeOv=0.0621
alpha-trim 3: PSNR=25.50, SSIM=0.7050, EdgeOv=0.6235
alpha-trim 5: PSNR=23.79, SSIM=0.5720, EdgeOv=0.2645
alpha-trim 7: PSNR=22.81, SSIM=0.4604, EdgeOv=0.0787
morph 3: PSNR=20.05, SSIM=0.5220, EdgeOv=0.3065
morph 5: PSNR=16.78, SSIM=0.3504, EdgeOv=0.0821
morph 7: PSNR=15.47, SSIM=0.3158, EdgeOv=0.0625
Best median size (by SSIM) = 3 (SSIM=0.6613)

```

/content/Picture1.png | speckle : Original vs BestMedian3



```

--- Image: /content/Picture1.png | Noise: gaussian
Baseline PSNR: 23.38, SSIM: 0.6221, EdgeOverlap: 0.7071
median 3: PSNR=25.93, SSIM=0.7217, EdgeOv=0.5836
median 5: PSNR=23.75, SSIM=0.5771, EdgeOv=0.2686
median 7: PSNR=22.78, SSIM=0.4657, EdgeOv=0.1087
median 9: PSNR=22.36, SSIM=0.4114, EdgeOv=0.0634
alpha-trim 3: PSNR=26.04, SSIM=0.7568, EdgeOv=0.6046
alpha-trim 5: PSNR=23.92, SSIM=0.5950, EdgeOv=0.2458
alpha-trim 7: PSNR=22.87, SSIM=0.4690, EdgeOv=0.0769
morph 3: PSNR=21.27, SSIM=0.5550, EdgeOv=0.3132
morph 5: PSNR=18.11, SSIM=0.3709, EdgeOv=0.0922
morph 7: PSNR=16.97, SSIM=0.3290, EdgeOv=0.0701
Best median size (by SSIM) = 3 (SSIM=0.7217)

```

/content/Picture1.png | gaussian : Original vs BestMedian3

